A5 PySpark performance

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

Contents

Track A: Databricks pipeline engineering 2
A5 PySpark performance 3
A5-L1 Lazy evaluation, and why every count() re-runs the plan 3
A5-L2 Built-in functions over Python UDFs 7
A5-L3 Joins, broadcast, partitions and reading the plan 11
Track A best-practice checklist 16
Appendix: solutions and answer keys 17

Track A: Databricks pipeline engineering

Build Databricks pipelines that are correct, fast, observable and tested.

A5 PySpark performance

Roadmap: Months 16 to 18: MLOps and deployment; Current work. Lazy evaluation, built-ins over UDFs, joins, partitioning, caching, explain plans.

A5-L1 Lazy evaluation, and why every count() re-runs the plan

Concept

Spark splits DataFrame operations into two kinds.

- **Transformations** build a plan and return a new DataFrame at once: `filter`, `select`, `withColumn`, `join`, `groupBy().agg()`. Nothing is read yet.
- **Actions** run the plan and return a result: `count()`, `collect()`, `show()`, `first()`, `toPandas()`, and every write.

This is **lazy evaluation**. It lets the optimiser see the whole plan before running it, so it can push filters into the file scan and skip columns you never use.

The cost is easy to miss: **each action runs the whole plan again**, from the source. Nothing is kept between actions unless you ask for it. A local test shows it plainly: a Python function counted its own calls inside a plan over 1,000 rows. After the first `count()`, it had run 1,000 times. After a second `count()`, 2,000.

Your reference data-contract validator calls `count()` more than ten times on the same uncached DataFrame: once for the row count, once per mandatory column, once per key and so on. Each call rereads the delivery from storage.

Three fixes, in order of preference:

1. **Combine the questions into one action.** One `agg` with many expressions answers them all in a single pass:

```
python df.agg( F.count("*").alias("rows"),
F.count_if(F.col("final_weight").isNull()).alias("null_weights"),
F.sum("final_weight").alias("weight_sum"), ).collect()[0]
```

1. **Materialise** an expensive intermediate result to a Delta table, then read that table for the follow-up actions.
2. **Cache** with `df.cache()` on classic clusters only. Serverless compute, which Free Edition uses, does not support cache APIs.

Why it matters

A contract validator checks a 40-million-row delivery with 23 separate `count()` calls, one per check. Each call rescans the source files in cloud storage. The validation step takes 38 minutes, longer than the transform it protects, and the team starts skipping it "for small redeliveries". A redelivery with duplicated keys then goes straight to silver.

Rewritten as one `agg` with 23 expressions, the same checks run in a single scan of about two minutes, and nobody has a reason to skip them.

Worked example: weak code

```
def check_delivery(df):
    results = {}
    results["rows"] = df.count()
    results["null_ids"] =
  df.filter(F.col("respondent_id").isNull()).count()
    results["null_weights"] =
  df.filter(F.col("final_weight").isNull()).count()
    results["distinct_ids"] =
  df.select("respondent_id").distinct().count()
    return results
```

1. **Line 3.** Action 1: full scan of the source.
2. **Line 4.** Action 2: a second full scan.
3. **Line 5.** Action 3: a third.
4. **Line 6.** Action 4: a fourth scan, plus a shuffle.

Worked example: strong code

```
def check_delivery(df: DataFrame) -> dict[str, int]:
    """Row, null and distinct counts for a delivery in one pass."""
    row = df.agg(
        F.count("*").alias("rows"),
        F.count_if(F.col("respondent_id").isNull()).alias("null_ids"),
        F.count_if(F.col("final_weight").isNull()).alias("null_weights"),
        F.count_distinct("respondent_id").alias("distinct_ids"),
    ).collect()[0]
    return row.asDict()
```

1. **Line 3.** One action. Spark computes every expression during the same scan.
2. **Line 5.** count_if counts rows where a condition holds, inside the aggregation.
3. **Line 7.** Distinct count in the same pass. For very large data, approx_count_distinct is cheaper.

Four questions, one scan. The strong version reads the delivery once, however many checks you add.

Recap card

- **Transformation or action?** Transformations (filter, select, join, groupBy().agg) build a plan. Actions (count, collect, show, write) run it.
- **What happens on the second count() of an uncached DataFrame?** The whole plan runs again from the source.
- **Many checks, one scan** One df.agg(...) with count, count_if, sum, min, max expressions.
- **Reusing an expensive result on serverless** Write it to a Delta table. cache() is not supported on serverless compute.

Practice

Exercise 1. Multiple choice (level 1)

Which line starts a Spark job?

- a) `df2 = df.filter(F.col("in_scope"))`
- b) `df3 = df2.withColumn("w2", F.col("final_weight") * 2)`
- c) `n = df3.count()`
- d) `df4 = df3.groupBy("wave_id").agg(F.sum("w2"))`

Why?

- r1) count() is an action. The other lines are transformations that only extend the plan.
- r2) Only the last line of a cell runs.

Exercise 2. Predict the output (level 2)

`tag` is a Python UDF that adds 1 to an accumulator each time it runs. `src` has 1,000 rows and is not cached. What does the last line print?

```
df = src.select(tag("id").alias("id")).filter(F.col("id") % 2 == 0)
df.count()
df.count()
print(calls.value)
```

Exercise 3. Fill the gap (level 2)

Count rows and null weights in a single pass.

```
stats = df._____(1)(
    F.count("*").alias("rows"),
    F._____(2)(F.col("final_weight").isNull()).alias("null_weights"),
).collect()[0]
```

Exercise 4. Refactor (level 4)

Refactor `table_health` to answer every question in **one** action. Keep the returned keys: `rows`, `null_ids`, `null_weights`, `min_weight`, `max_weight`.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```python
# Row count, null counts and weight range in one pass over df.
def table_health(
    df: DataFrame,
) -> dict[str, float | int | None]: ...
```

Starter code:

```python
def table_health(df):
    return {
        "rows": df.count(),
        "null_ids": df.filter(F.col("respondent_id").isNull()).count(),
        "null_weights": df.filter(F.col("final_weight").isNull()).count(),
        "min_weight": df.agg({"final_weight": "min"}).collect()[0][0],
        "max_weight": df.agg({"final_weight": "max"}).collect()[0][0],
    }
```

Exercise 5. Architecture design (level 5)

You run the silver QA notebook on Databricks Free Edition (serverless). An expensive joined DataFrame feeds six different checks. Choose the approach for each situation.

Options: Combine them into one agg, Write the joined data to a Delta table and read it back, df.cache(), Leave each check as its own action.

1. Four checks are simple counts and sums over the joined data
2. Two checks need different groupings and joins over the same expensive result
3. A colleague suggests cache() to speed it all up on serverless

Sources

- Each action re-executes the plan: a counting UDF ran 1,000 times after one count and 2,000 after two, on 1,000 rows. (verified: Run on PySpark 4.0.4 in this repository.)
- F.count_if and F.count_distinct exist in PySpark 4.0. (verified: Run on PySpark 4.0.4 in this repository.)
- Serverless compute does not support DataFrame or SQL cache APIs. (verified: https://docs.databricks.com/aws/en/compute/serverless/limitations (checked 2026-09-25))

A5-L2 Built-in functions over Python UDFs

Concept

A **Python UDF** runs your Python function once per row. Spark must send each value from the JVM to a Python worker process, run the function, and send the result back. The physical plan shows this as a `BatchEvalPython` step.

A **built-in function** from `pyspark.sql.functions` runs inside the JVM, in Spark's generated code, with no round trip.

Measured on 2 million strings (PySpark 4.0.4, two local cores):

Approach	Time
`F.upper(col)`	0.30 s
pandas UDF (`ArrowEvalPython`)	1.16 s
Python UDF (`BatchEvalPython`)	1.82 s

The Python UDF was six times slower, and the gap grows with cluster size and data volume.

Speed is one reason. There are three more:

- **Nulls.** Built-ins return NULL for NULL input. A UDF receives `None` and must handle it. `lambda s: s.strip()` fails the whole job with a `PythonException` on the first NULL.
- **Optimisation.** The optimiser cannot see inside a UDF, so it cannot push a filter on the UDF's output into the file scan.
- **Readability.** `F.when(...).when(...)` states a rule in one expression a reviewer can check.

Most UDFs in pipelines replace functions that already exist: `F.upper`, `F.trim`, `F.regexp_replace`, `F.regexp_extract`, `F.split`, `F.to_date`, `F.coalesce`, `F.when`, `F.concat_ws`, `F.lpad`. Search the functions reference before you write one.

When no built-in fits, prefer a **pandas UDF**: it moves data in Arrow batches and runs vectorised pandas code. Use a row-by-row Python UDF only as the last resort, and handle `None` first.

Why it matters

A silver job cleans postcodes with a Python UDF: `lambda s: s.replace(" ", "").upper()`. It runs for months on deliveries where every postcode is present. Then a supplier sends a wave with 40 blank postcodes, which arrive as NULL. The first NULL raises `AttributeError: 'NoneType' object has no`

`attribute 'replace'` inside a Python worker, and the whole silver job fails at 02:00.

`F.upper(F.regexp_replace("postcode", r"\s+", ""))` returns NULL for NULL, runs faster, and would never have failed.

Worked example: weak code

```
1  from pyspark.sql.functions import udf
2
3
4  @udf("string")
5  def clean_postcode(s):
1      return s.replace(" ", "").upper()
7
8
9  @udf("string")
10 def age_band(age):
2      if age < 35:
12         return "16-34"
13     if age < 55:
14         return "35-54"
15     return "55+"
16
17
3  silver = bronze.withColumn("postcode",
       clean_postcode("postcode")).withColumn("age_group", age_band("age"))
```

1. **Line 6.** Crashes on NULL: None has no replace method.
2. **Line 11.** In Python, comparing None with 35 raises TypeError, so a NULL age crashes the job too.
3. **Line 18.** Two round trips to Python per row, invisible to the optimiser.

Worked example: strong code

```
1  from pyspark.sql import Column
2  from pyspark.sql import functions as F
3
4
1  def clean_postcode(col: Column) -> Column:
6      """Remove whitespace and upper-case. NULL stays NULL."""
2      return F.upper(F.regexp_replace(col, r"\s+", ""))
8
9
10 def age_band(age: Column) -> Column:
11     """16-34, 35-54 or 55+. NULL age gives NULL."""
12     return (
13         F.when(age < 35, "16-34")
14         .when(age < 55, "35-54")
3          .when(age.isNotNull(), "55+")
16     )
```

```
17
18
19 silver = bronze.withColumns({
20     "postcode": clean_postcode(F.col("postcode")),
21     "age_group": age_band(F.col("age")),
22 })
```

1. **Line 5.** A function that builds a Column expression. Reusable and testable, with no UDF.
2. **Line 7.** Built-ins pass NULL through as NULL.
3. **Line 15.** Explicit: a NULL age stays NULL instead of falling into 55+.

The strong version runs in the JVM, survives NULLs and reads as a rule. Functions that return Column expressions give you the reuse that UDFs promised.

Recap card

- **Why are Python UDFs slow?** Every value crosses from the JVM to a Python worker and back. The plan shows BatchEvalPython.
- **Measured cost on 2 million strings** Built-in 0.30 s, pandas UDF 1.16 s, Python UDF 1.82 s (PySpark 4.0.4, local).
- **What does a Python UDF receive for NULL?** None. s.strip() then raises and fails the job. Built-ins return NULL for NULL.
- **Reusable logic without a UDF** A Python function that takes and returns Column expressions, built from F.when, F.regexp_replace and friends.

Practice

Exercise 1. Multiple choice (level 1)

You need to trim and upper-case a string column on 50 million rows. Which approach is fastest?

- a) A Python UDF: `udf(lambda s: s.strip().upper() if s else None)`
- b) A pandas UDF using `s.str.strip().str.upper()`
- c) `F.upper(F.trim(F.col("s")))`
- d) `collect()` the column and process it in Python

Why?

- r1) Built-ins run inside the JVM with no data transfer to Python.
- r2) Built-ins run on the driver, which is faster than executors.

Exercise 2. Predict the output (level 2)

What does this print?

```
df = spark.createDataFrame([(" ab 1 ",), (None,)], "s STRING")
out = df.select(F.upper(F.regexp_replace(F.col("s"), r"\s+",
    "")).alias("u"))
print([r.u for r in out.collect()])
```

Exercise 3. Fill the gap (level 3)

Replace a UDF that pads respondent IDs to 8 characters with leading zeros and extracts the 2-letter region prefix from a code such as `NW-1234`.

```
df = df.withColumns({
    "respondent_id": F._____(1)(F.col("respondent_id"), 8, "0"),
    "region": F._____(2)(F.col("area_code"), r"^([A-Z]{2})-", 1),
})
```

Exercise 4. Refactor (level 4)

Replace the UDF with a function `age_band(age: Column) -> Column` built from `F.when`. Keep the bands: under 35 is `16-34`, under 55 is `35-54`, anything else `55+`. A NULL age must give NULL.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# 16-34, 35-54 or 55+. NULL age gives NULL.
def age_band(age: Column) -> Column: ...
```

Starter code:

```
@udf("string")
def age_band(age):
    if age < 35:
        return "16-34"
    if age < 55:
        return "35-54"
    return "55+"
```

Exercise 5. Spot the bug (level 5)

This job ran for months, then failed on a wave where some postcodes are blank. Tap the line that fails, then rewrite `clean_postcodes` without a UDF.

```
from pyspark.sql.functions import udf


@udf("string")
def _clean(s):
    return s.replace(" ", "").upper()


def clean_postcodes(df: DataFrame) -> DataFrame:
    """Remove spaces from postcodes and upper-case them."""
    return df.withColumn("postcode", _clean("postcode"))
```

Write down the line number of the bug. Then write the fix.

Sources

- Timings: built-in 0.30 s, pandas UDF 1.16 s, Python UDF 1.82 s on 2 million strings; plans show BatchEvalPython and ArrowEvalPython. (verified: Measured on PySpark 4.0.4, local[2], in this repository.)
- Built-ins return NULL for NULL input; a Python UDF calling a string method on None raises PythonException. (verified: Run on PySpark 4.0.4 in this repository.)

A5-L3 Joins, broadcast, partitions and reading the plan

Concept

A join between two large tables needs a **shuffle**: Spark moves rows across the network so equal keys meet on the same executor, then sorts and merges them. The plan shows this as `SortMergeJoin` with `Exchange` steps. Shuffles are the most expensive thing Spark does.

When one side is small, such as a lookup of 200 region codes, Spark can **broadcast** it: send a full copy to every executor and join locally, with no shuffle of the large side. The plan shows `BroadcastHashJoin`.

- Spark broadcasts automatically when it estimates a side is under `spark.sql.autoBroadcastJoinThreshold`, 10 MB by default in Spark 4.0.
- Adaptive query execution (AQE), on by default, can switch a join to broadcast at run time once it sees real sizes.
- You can force it with `F.broadcast(small_df)` or `small_df.hint("broadcast")`. Do this for lookups you know are small. Never broadcast something that grows with your data.

Read the plan before you tune. `df.explain()` prints the physical plan. `df.explain(mode="formatted")` adds details per step, including `PushedFilters`, which confirm that a filter reached the file scan.

Partitions decide parallelism.

- `repartition(n)` shuffles to exactly `n` partitions. `coalesce(n)` merges partitions without a shuffle, and only reduces the count.
- `repartition(1)` or `coalesce(1)` before a large write puts all the work on one task. Avoid it outside tiny outputs.

- For table layout on Unity Catalog managed tables, prefer liquid clustering over `partitionBy`. If you do partition a table, choose a low-cardinality column such as `wave_id`, never an ID.

Caching: `df.cache()` helps on classic clusters when several actions reuse a result. Serverless compute does not support it. There, write the intermediate result to a Delta table.

Why it matters

A gold job joins 40 million respondent rows to a 200-row region lookup. Auto-broadcast does not kick in, because the lookup is read through a view whose size Spark overestimates. Every run shuffles all 40 million rows across the cluster, and the join takes 22 minutes.

`explain()` shows `SortMergeJoin` with two `Exchange` steps. Wrapping the lookup in `F.broadcast()` turns it into `BroadcastHashJoin`, and the join drops to under two minutes. Five minutes reading the plan saved twenty minutes on every run.

Worked example: weak code

```
1  def enrich(fact, regions):
2      regions_local = regions.collect()
3      lookup = {r.region_code: r.region for r in regions_local}
4      label = F.udf(lambda code: lookup.get(code), "string")
5      out = fact.withColumn("region", label("region_code"))
6      return out.repartition(1)
```

1. **Line 2.** Pulls the lookup to the driver to build a Python dict. A join does this job.
2. **Line 4.** A Python UDF per row, where a broadcast join would stay in the JVM.
3. **Line 6.** All 40 million rows squeezed into one partition, so one task does all the writing.

Worked example: strong code

```
1  def enrich_with_regions(fact: DataFrame, regions: DataFrame) -> DataFrame:
2      """Add region names to fact rows. regions is a small lookup, so
       broadcast it.
3
4      A left join keeps every fact row, even with an unknown region code.
5      """
6      return fact.join(F.broadcast(regions), on="region_code", how="left")
7
8
9  def show_plan(df: DataFrame) -> None:
10     """Print the formatted physical plan: check for BroadcastHashJoin."""
```

```
❸     df.explain(mode="formatted")
```

1. **Line 6.** A left join keeps fact rows with unknown codes. An inner join would silently drop them.
2. **Line 6.** Broadcast the small side. The plan shows BroadcastHashJoin and no shuffle of the fact table.
3. **Line 11.** Confirm the strategy and the pushed filters before and after tuning.

One broadcast join replaces a collect, a dict and a UDF, keeps the work distributed, and shows up clearly in the plan.

Recap card

- **Join strategy for a large table and a small lookup** Broadcast the lookup: F.broadcast(small). The plan shows BroadcastHashJoin.
- **Default auto-broadcast threshold in Spark 4.0** 10 MB (spark.sql.autoBroadcastJoinThreshold). AQE can also switch to broadcast at run time.
- **repartition versus coalesce** repartition(n) shuffles to n partitions. coalesce(n) merges without a shuffle and only reduces.
- **How do you confirm a filter reached the file scan?** df.explain(mode="formatted") and look for PushedFilters on the scan.

Practice

Exercise 1. Multiple choice (level 1)

You join a 40-million-row respondent table with a 200-row region lookup. Which strategy should the plan show?

- a) SortMergeJoin, shuffling both sides
- b) BroadcastHashJoin, with the lookup sent to every executor
- c) Collect the lookup and use a Python UDF
- d) A cross join, then a filter

Why?

- r1) Broadcasting the small side avoids shuffling the large side.
- r2) Broadcast joins always use less memory.

Exercise 2. Predict the output (level 2)

Which join node name appears in the physical plan printed by this code on Spark 4.0?

```
1 joined = fact.join(F.broadcast(regions), "region_code", "left")
2 joined.explain()
```

Exercise 3. Fill the gap (level 3)

Broadcast the lookup, keep every fact row, then print the detailed plan.

```
joined = fact.join(F._____(1)(regions), on="region_code", how="_____(2)")
joined.explain(mode="_____(3)")
```

Exercise 4. Spot the bug (level 4)

Writing gold takes 45 minutes on a 16-core cluster, and the Spark UI shows one busy task. Tap the line, then fix the function.

```
def write_gold(metrics: DataFrame, table: str) -> None:
    """Write gold metrics for the wave."""
    (
        metrics.repartition(1)
        .write.format("delta")
        .mode("append")
        .saveAsTable(table)
    )
```

Write down the line number of the bug. Then write the fix.

Exercise 5. Write the function (level 5)

Write `enrich_with_regions(fact, regions)`. It left-joins `fact` to the small `regions` lookup on `region_code`, broadcasting the lookup. Then write `unmatched_codes(enriched)` that returns the number of rows where `region` is NULL, using a single aggregation.

Shape of the answer. The reference solution defines these names. Write the bodies yourself.

```
# Left-join fact rows to the small regions lookup, broadcasting the
    lookup.
def enrich_with_regions(
    fact: DataFrame,
    regions: DataFrame,
) -> DataFrame: ...

# Rows whose region_code had no match in the lookup.
def unmatched_codes(enriched: DataFrame) -> int: ...
```

Starter code:

```
from pyspark.sql import DataFrame
from pyspark.sql import functions as F


def enrich_with_regions(fact: DataFrame, regions: DataFrame) -> DataFrame:
    ...


def unmatched_codes(enriched: DataFrame) -> int:
```

```
10     ...
```

Practical: Read the plan: sort-merge join versus broadcast join

Platform: Databricks Free Edition. Run this in Databricks Free Edition. It builds a fact table and a small lookup, then prints the physical plan with and without a broadcast hint, so you can find `SortMergeJoin`, `BroadcastHashJoin` and `Exchange` yourself. On serverless, Spark settings you can change are limited, so the notebook compares plans rather than timings.

Download the notebook `A5-L3.ipynb` from the app.

Sources

- F.broadcast and hint('broadcast') give BroadcastHashJoin; with broadcasting off, SortMergeJoin; autoBroadcastJoinThreshold defaults to 10 MB; AQE is on by default; explain supports mode='formatted' with PushedFilters. (verified: Run on PySpark 4.0.4 in this repository.)
- Databricks recommends liquid clustering over partitioning for UC managed tables; serverless does not support cache APIs. (verified: https://docs.databricks.com/aws/en/tables/clustering and https://docs.databricks.com/aws/en/compute/serverless/limitations (checked 2026-09-25))

Track A best-practice checklist

☐ Every function has type hints, a docstring and one job.

☐ No SparkSession is created at import time. Functions receive spark as a parameter.

☐ Functions are imported as F: `from pyspark.sql import functions as F`.

☐ Table names, column maps and thresholds come from config or registry tables.

☐ Every table states its grain in a comment or table property.

☐ Bronze appends. Silver deduplicates the source, then MERGEs. Gold aggregates by every grain column.

☐ Every run writes one record to the run log table with an explicit schema.

☐ CRITICAL checks raise. WARN checks log and continue.

☐ Every division has a guard for zero and null.

☐ Counts come from one agg pass, not repeated count() calls.

☐ Incremental loads compare source and target on the same keys.

☐ Row filters use a SQL UDF with SET ROW FILTER. Grants go to groups.

☐ Tests run on a local SparkSession in CI before every merge.

Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

A5-L1 Lazy evaluation, and why every count() re-runs the plan

A5-L1 exercise 1

Hints. (1) Which call returns a plain Python value instead of a DataFrame? (2) Transformations return DataFrames. (3) count() returns an int, so it must run the plan.

Answer. c) `n = df3.count()` Because: r1) count() is an action. The other lines are transformations that only extend the plan.

`count()` returns a number, so Spark must execute the plan. The other lines return DataFrames and only describe work.

Common wrong answers.

- `a`: filter is a transformation. It returns a new DataFrame without reading any data.
- `b`: withColumn is a transformation. Nothing runs yet.
- `d`: groupBy().agg() is still a transformation. It runs only when an action such as show or write asks for the result.
- `c|r2`: Right line, wrong reason. Every line runs as Python. Only actions make Spark execute the plan.

A5-L1 exercise 2

Hints. (1) How many actions are there? (2) Does Spark keep results between actions without cache? (3) Two full runs over 1,000 rows.

Answer.

```
1 2000
```

Nothing is kept between actions, so each `count()` runs the UDF on all 1,000 rows. This was measured locally on PySpark 4.0.4.

Common wrong answers.

- `1000`: Each action re-runs the whole plan, UDF included. Two counts mean two full evaluations.

- `500`: The filter comes after the UDF, so the UDF runs on all 1,000 rows each time.
- `0`: Defining df runs nothing, but each count() is an action that executes the plan.

A5-L1 exercise 3

Hints. (1) The DataFrame method that takes several aggregate expressions. (2) The function that counts rows where a condition is true. (3) `agg` and `count_if`.

Answer.

```
stats = df.agg(
    F.count("*").alias("rows"),
    F.count_if(F.col("final_weight").isNull()).alias("null_weights"),
).collect()[0]
```

`df.agg(...)` evaluates every expression in one pass. `F.count_if(cond)` counts rows where `cond` is true.

Common wrong answers.

- g1 = `select`: `select` with aggregate functions also works, but `agg` is the idiomatic call for a single aggregated row.
- g2 = `count`: `F.count` counts non-null values of an expression, and `isNull()` is never null. Every row would count. Use `count_if`.
- g2 = `sum`: `F.sum` of a boolean fails. You would need `.cast("int")` first. `count_if` says what you mean.

A5-L1 exercise 4

Hints. (1) Every value can be an aggregate expression. (2) `F.count("*")`, `F.count_if(...)`, `F.min(...)`, `F.max(...)`. (3) Put all five in one `df.agg(...)`, collect once, and return `row.asDict()`.

Answer.

```
def table_health(df: DataFrame) -> dict[str, float | int | None]:
    """Row count, null counts and weight range in one pass over df."""
    row = df.agg(
        F.count("*").alias("rows"),
        F.count_if(F.col("respondent_id").isNull()).alias("null_ids"),
        F.count_if(F.col("final_weight").isNull()).alias("null_weights"),
        F.min("final_weight").alias("min_weight"),
        F.max("final_weight").alias("max_weight"),
    ).collect()[0]
    return row.asDict()
```

Five aggregate expressions in one `agg` produce one row in one scan. The original ran five separate jobs over the same data.

A5-L1 exercise 5

Hints. (1) Simple aggregates can share one pass. (2) Different shapes need the data stored somewhere reusable. (3) Serverless has no cache APIs.

Answer.

- Combine every count and sum into one `agg`. One scan answers them all.
- For checks that need different shapes, write the expensive result to a Delta table once and read it back.
- Serverless compute does not support cache APIs, so `cache()` is not an option there.

Combine what you can into one action. Store an expensive result in Delta when several differently shaped queries need it. Keep cache() for classic clusters.

Common wrong answers.

- several_counts = `repeat`: Each action re-runs the join. Four checks would run the expensive join four times.
- many_shapes = `cache`: On serverless compute, cache APIs are not supported. Materialise to a Delta table instead.
- cache_serverless = `use`: Databricks documents that DataFrame and SQL cache APIs are not supported on serverless compute.

A5-L2 Built-in functions over Python UDFs

A5-L2 exercise 1

Hints. (1) Which option never leaves the JVM? (2) Are trim and upper available as built-ins? (3) `F.trim` and `F.upper`.

Answer. c) `F.upper(F.trim(F.col("s")))` Because: r1) Built-ins run inside the JVM with no data transfer to Python.

Built-ins run in Spark's generated JVM code on the executors. UDFs of either kind pay to move data to Python and back.

Common wrong answers.

- `a`: A Python UDF moves every value to a Python process and back. It measured about six times slower than a built-in.

- `b`: A pandas UDF beats a Python UDF because it uses Arrow batches, but it still leaves the JVM. A built-in wins.
- `d`: collect() pulls 50 million values onto one machine. It is the slowest option and can crash the driver.
- `c|r2`: Right choice, wrong reason. Built-ins run on the executors, like everything else. They win by staying in the JVM.

A5-L2 exercise 2

Hints. (1) What does `\s+` match? (2) What do built-ins return for NULL input? (3) Whitespace removed, then upper-cased. NULL stays NULL.

Answer.

```
['AB1', None]
```

The regex removes all whitespace, `upper` capitalises, and the NULL row stays NULL. No special handling needed.

Common wrong answers.

- `[' AB 1 ', None]`: `regexp_replace` with `\s+` removes every run of whitespace, including the inner space.
- `PythonException`: Built-ins handle NULL. Only a Python UDF would raise on the None row.
- `['AB1', '']`: NULL input gives NULL output, not an empty string.

A5-L2 exercise 3

Hints. (1) Padding on the left has a function named after that side. (2) Pulling a capture group out of a string uses a regex function. (3) `lpad` and `regexp_extract`.

Answer.

```
df = df.withColumns({
    "respondent_id": F.lpad(F.col("respondent_id"), 8, "0"),
    "region": F.regexp_extract(F.col("area_code"), r"^([A-Z]{2})-", 1),
})
```

`F.lpad(col, 8, "0")` adds leading zeros. `F.regexp_extract(col, pattern, 1)` returns capture group 1, or an empty string when there is no match.

Common wrong answers.

- g1 = `rpad`: `rpad` pads on the right. Leading zeros go on the left: `lpad`.
- g2 = `regexp_replace`: `regexp_replace` substitutes text. To pull out a capture group, use `regexp_extract` with the group index.

- g2 = `substring`: `substring` takes positions, not a pattern, and cannot check the hyphen. `regexp_extract` with group 1 matches the rule exactly.

A5-L2 exercise 4

Hints. (1) The function takes a Column and returns a Column expression. (2) `F.when(age < 35, "16-34").when(age < 55, "35-54")`. (3) Finish with `.when(age.isNotNull(), "55+")`, not `.otherwise`.

Answer.

```
1  from pyspark.sql import Column
2  from pyspark.sql import functions as F
3
4
5  def age_band(age: Column) -> Column:
6      """16-34, 35-54 or 55+. NULL age gives NULL."""
7      return (
8          F.when(age < 35, "16-34")
9          .when(age < 55, "35-54")
10         .when(age.isNotNull(), "55+")
11     )
```

`F.when` chains evaluate in order. Without `otherwise`, an unmatched row is NULL, and `age < 35` is NULL for a NULL age, so the final `isNotNull()` guard keeps NULL ages NULL.

A5-L2 exercise 5

Hints. (1) What does the UDF receive for a NULL postcode? (2) `None.replace` raises AttributeError. (3) Replace the UDF with `F.upper(F.regexp_replace(...))`.

Answer. The bug is on line 6.

Fixed version:

```
1  from pyspark.sql import DataFrame
2  from pyspark.sql import functions as F
3
4
5  def clean_postcodes(df: DataFrame) -> DataFrame:
6      """Remove spaces from postcodes and upper-case them. NULL stays
       NULL."""
7      return df.withColumn("postcode",
       F.upper(F.regexp_replace(F.col("postcode"), r"\s+", "")))
```

For a NULL postcode the UDF receives `None`, and `None.replace` raises inside the Python worker, failing the job. Built-ins pass NULL through, and run faster.

Common wrong answers.

- Line 4: The decorator makes it a UDF, which is slow, but the crash happens where a value is used.
- Line 11: The call is fine. The failure is inside the function, on a NULL value.

A5-L3 Joins, broadcast, partitions and reading the plan

A5-L3 exercise 1

Hints. (1) Which side is small? (2) What is the most expensive step in a join of two large tables? (3) Copy the small side to every executor.

Answer. b) BroadcastHashJoin, with the lookup sent to every executor Because: r1) Broadcasting the small side avoids shuffling the large side.

With a tiny lookup, broadcast it. The large table stays where it is, and each executor joins locally.

Common wrong answers.

- `a`: A sort-merge join shuffles all 40 million rows across the network to meet 200 lookup rows. Broadcasting the lookup avoids that.
- `c`: That moves the join into a slow Python UDF. A broadcast join does the same job in the JVM.
- `d`: A cross join multiplies 40 million by 200 before filtering: 8 billion rows.
- `b|r2`: Right strategy, wrong reason. Broadcasting costs memory on every executor. It wins by avoiding the shuffle, and only when the broadcast side is small.

A5-L3 exercise 2

Hints. (1) The hint names the strategy. (2) An equality join with a broadcast side uses a hash table. (3) Broadcast + Hash + Join.

Answer.

```
1 BroadcastHashJoin
```

`F.broadcast` makes Spark build a hash table from `regions` on every executor. The plan shows `BroadcastHashJoin` with `LeftOuter`.

Common wrong answers.

- `SortMergeJoin`: The broadcast hint tells Spark to broadcast `regions`, so the plan uses BroadcastHashJoin.
- `BroadcastNestedLoopJoin`: That node appears for joins without an equality condition. This join is on region_code, so it is a hash join.

A5-L3 exercise 3

Hints. (1) The function names the join strategy. (2) Keep all rows from the left side. (3) The explain mode that splits the plan into numbered, detailed steps.

Answer.

```
1 joined = fact.join(F.broadcast(regions), on="region_code", how="left")
2 joined.explain(mode="formatted")
```

`F.broadcast(regions)` forces a broadcast hash join, `how="left"` keeps every fact row, and `mode="formatted"` prints a detailed plan with pushed filters.

Common wrong answers.

- g2 = `inner`: An inner join drops fact rows whose region code is missing from the lookup, without any warning. Use a left join.
- g3 = `verbose`: explain modes are simple, extended, codegen, cost and formatted. `formatted` gives the per-step details.
- g1 = `cache`: Caching does not change the join strategy. `F.broadcast` does.

A5-L3 exercise 4

Hints. (1) How many partitions does the write use? (2) One partition means one task. (3) Remove `.repartition(1)`.

Answer. The bug is on line 4.

Fixed version:

```
1 def write_gold(metrics: DataFrame, table: str) -> None:
2     """Write gold metrics for the wave."""
3     metrics.write.format("delta").mode("append").saveAsTable(table)
```

`repartition(1)` funnels every row into one task, so 15 of 16 cores sit idle. Remove it. For file sizes, rely on OPTIMIZE, liquid clustering or predictive optimization.

Common wrong answers.

- Line 6: Append is the right mode here. The slowness comes from how many tasks do the writing.
- Line 5: Delta is the right format. Look at the partition count.

A5-L3 exercise 5

Hints. (1) `fact.join(F.broadcast(regions), on=..., how=...)`. (2) A left join keeps unknown codes, with region NULL. (3) `enriched.agg(F.count_if(F.col("region").isNull())).collect()[0][0]`.

Answer.

```python
from pyspark.sql import DataFrame
from pyspark.sql import functions as F


def enrich_with_regions(fact: DataFrame, regions: DataFrame) -> DataFrame:
    """Left-join fact rows to the small regions lookup, broadcasting the lookup."""
    return fact.join(F.broadcast(regions), on="region_code", how="left")


def unmatched_codes(enriched: DataFrame) -> int:
    """Rows whose region_code had no match in the lookup."""
    return int(enriched.agg(F.count_if(F.col("region").isNull())).collect()[0][0])
```

The broadcast left join keeps every fact row and never shuffles the fact table. Counting NULL regions afterwards turns silently unknown codes into a number your QA can check.